Spanning trees and complexity theory

Guillaume Dalle (ENPC, CERMICS)

REOP - Class 4 (20/10/2021)

Contents

1 Minimum spanning trees **1**
1.1 Definitions . . . 1
1.2 Greedy construction . . . 1
1.2.1 Kruskal's algorithm . . . 1
1.2.2 Correctness argument . . . 2
1.2.3 Remarks . . . 2
1.3 Spanning tree polytope . . . 2

2 Introduction to complexity theory **3**
2.1 Decision and optimisation problems . . . 3
2.2 Complexity classes . . . 3
2.2.1 Why is it important? . . . 3
2.2.2 P and NP . . . 3
2.2.3 NP-completeness . . . 4
2.2.4 Optimisation problems . . . 4

You will find solutions for this week's homework (exercises 6.10 and 6.13) in the complete lecture notes on Educnet.

1 Minimum spanning trees

1.1 Definitions

A spanning subgraph $\mathcal{H} = (V, F)$ of $\mathcal{G} = (V, E)$ is a set of edges incident to every vertex. If $\mathcal{H}$ is a also a tree (connected without cycles), we talk of a spanning tree.

Minimum spanning tree (MST) problem:

- Input: An undirected connected graph $\mathcal{G} = (V, E)$, a weight function $c : E \to \mathbb{R}$
- Output: A spanning tree $\mathcal{T} = (V, T)$ of minimum weight

1.2 Greedy construction

1.2.1 Kruskal's algorithm

1. Sort E by increasing edge weight: $E = \{e_1, ..., e_m\}$ with $c(e_i) \leq c(e_{i+1})$
2. Start with $F_0 = \emptyset$
3. For $i = 1, ..., n$: if $F_{i-1} \cup \{e_i\}$ has no cycles, set $F_i = F_{i-1} \cup \{e_i\}$, otherwise set $F_i = F_{i-1}$.
4. Return $\mathcal{T} = (V, F_m)$.

Apply it on Exercise 4.2.

1.2.2 Correctness argument

At any time during the algorithm, $\mathcal{F}_i = (V, F_i)$ is a forest because we ensure that it remains without cycles. So $\mathcal{T}$ is a forest.

If $\mathcal{T} = (V, F_m)$ is not connected (in particular, if it has isolated vertices), then let V_1 and V_2 be two connected components in $\mathcal{T}$. Since $\mathcal{G}$ is connected, there is an edge $e \in E$ between V_1 and V_2: this edge does not belong to $\mathcal{T}$ and does not create a cycle, hence it should have been added at some point in the algorithm. We obtain a contradiction: $\mathcal{T}$ is a connected forest on V, aka a spanning tree.

To prove that $\mathcal{T}$ has minimum weight among all spanning trees, we can use the following loop invariant: *for all i, there is a minimum spanning tree $\mathcal{T}_i = (V, T_i)$ such that $\mathcal{F}_i = (V, F_i)$ is a subgraph of $\mathcal{T}_i$.* This will allow us to conclude that $\mathcal{T} = \mathcal{T}_m$ (since both are trees with the same number of vertices).

Loop invariant proof (do it as homework, and use a drawing!)

Let's check that the invariant mentioned above holds after iteration i. If we don't add edge e_i to F_{i-1}, nothing happens, and since $F_i = F_{i-1} \subset T_{i-1}$, we can take $T_i := T_{i-1}$ to be a MST containing F_i. If we add e_i to F_{i-1} and e_i is already in T_{i-1}, we can also take $T_i := T_{i-1}$. If we add e_i to F_{i-1} but e_i is not in T_{i-1}, then we must be more clever. Let $e_i = (u_i, v_i)$, and let X be the connected component of $\mathcal{F}_{i-1}$ containing u_i. We can prove that:

- X does not contain v_i (because $e_i \notin F_{i-1}$ and adding it does not create a cycle)
- The MST T_{i-1} contains exactly one path between u_i and v_i (because it is a tree). This path crosses the cut $\delta(X, V \backslash X)$ using an edge $f \neq e_i$ (because $e_i \notin T_{i-1}$).
- The cut $\delta(X, V \backslash X)$ is disjoint from F_{i-1} (otherwise the connected component X could be extended).
- The cut $\delta(X, V \backslash X)$ does not contain any e_j with $j < i$ (because such an edge is either in F_{i-1} or creates a cycle in F_{i-1}, which would require some edge of $\delta(X, V \backslash X)$ to be in F_{i-1})
- So $f = e_j$ with $e_j > e_i$, which means $c(f) \geq c(e_i)$. As a consequence, if we define $T_i = T_{i-1} \backslash \{f\} \cup \{e_i\}$, then we have $c(T_i) \leq c(T_{i-1})$.
- The subgraph T_i is still a spanning tree (because $T_i \cup \{e_i\}$ has exactly one cycle containing e_i and f)

1.2.3 Remarks

Kruskal's algorithm is greedy: quite surprising that it returns an optimal solution! Prim's algorithm is another example of greedy algorithm for the MST: instead of uniting forests, it grows a single tree.

Implementing Kruskal's algorithm is non-trivial: naively checking whether e_i creates a cycle in F_{i-1} would be very inefficient. Standard implementations use a specific data structure called "union-find" to store the disjoint forests and merge them.

1.3 Spanning tree polytope

The MST problem can be formulated as a Mixed Integer Linear Program:

$$\min \sum_{e \in E} c(e) x_e \quad \text{s.t.} \begin{cases} \sum_{e \in E} x_e = |V| - 1 & \\ \sum_{e \in E[X]} x_e \leq |X| - 1 & \forall X \subset V, X \notin \{\emptyset, V\} \\ x_e \in \{0, 1\} & \forall e \in E \end{cases}$$

Given an optimal solution x^* to this program, we can construct a subgraph (V, E_{x^*}) with $E_{x^*} = \{e \in E : x^*_e = 1\}$.

The second constraint imposes the absence of cycle in any subgraph, which ensures we get a forest. The first constraint tells us that the forest is maximal, i.e. a (spanning) tree. The objective is there to ensure minimality.

However, this MILP is not easy to solve, for two reasons:

- The integrality constraint $x_e \in \{0, 1\}$. This is solved by Theorem 4.4: we can replace the constraint by $x_e \geq 0$ and still be sure that basic optimal solutions (returned by the simplex) are binary-valued.
- The exponential number of constraints. This is taken care of by Exercise 4.4, which shows how to decide efficiently whether any constraint is broken. Counterintuitively, this means we can solve the LP efficiently, at least in theory.

In the standard setting, there is no need for this LP since Kruskal's algorithm is faster. However, it is very useful when additional constraints or objectives are involved.

2 Introduction to complexity theory

See Session 1 for the intuitive definitions of "problem" and "algorithm."

There are undecidable problems, such as the Halting problem (deciding whether an algorithm will terminate or not on a given input). Here we focus on decidable problems and study their complexity.

2.1 Decision and optimisation problems

Formally, anything we can give to a computer can be described as a word x from a language X. A decision problem is a couple (X, Y) where

- X is a language called the input ("decidable" in polynomial time)
- its elements x are called instances
- $Y \subset X$ contains all instances for which the answer is "yes"
- $X \backslash Y$ contains all instances for which the answer is "no"

A solution algorithm is a function $f : X \to \{\texttt{yes}, \texttt{no}\}$ such that $f(Y) = \{\texttt{yes}\}$ and $f(X \backslash Y) = \{\texttt{no}\}$. The algorithm f runs in polynomial time if there is a polynomial P such that its number of steps in its execution on instance x can be bounded by $P(\texttt{size}(x))$. Here $\texttt{size}(x)$ is the length of the word x in binary!

2.2 Complexity classes

2.2.1 Why is it important?

The complexity class determines the method we can try:

- Fast exact algorithms for "easy" problems
- Fast approximate or heuristic algorithms for "hard" problems

Easy	Hard
Eulerian cycle	Hamiltonian cycle
Shortest path	Longest path
2-SAT	3-SAT
Minimum spanning tree	Steiner tree
Train shunting (night)	Train shunting (day)

2.2.2 P and NP

The class $\mathcal{P}$ contains all decision problems for which there is a polynomial algorithm that returns a solution.

The class $\mathcal{NP}$ (for "Nondeterministic Polynomial") contains all decision problems for which there is a polynomial algorithm that verifies a solution, using a so-called certificate. We don't need to be able to generate a certificate in polynomial time, just to check it against the instance.

Exercise : prove that $\mathcal{P} \subset \mathcal{NP}$.

Remarks:

- Why polynomial time (Cobham's thesis)? Independent of the underlying computer model.
- Not independent of the encoding: binary vs unary.
- Why no mention of memory? On a Turing machine, memory is bounded by time.

Exercise: Devise a dynamic programming algorithm for the knapsack problem. Is it in $\mathcal{P}$?

2.2.3 NP-completeness

A reduction of a problem $P = (X, Y)$ to $P' = (X', Y')$ is a mapping $r : X \to X'$ such that $x \in Y \iff r(x) \in Y'$. The reduction is called polynomial if r can be computed in polynomial time.

A decision problem P is $\mathcal{NP}$-complete if

1. P is in $\mathcal{NP}$
2. Any decision problem P' in $\mathcal{NP}$ reduces polynomially to P

Since mathematicians have already compiled a large list of $\mathcal{NP}$-complete problems, to prove the second point we only need to reduce a known $\mathcal{NP}$-complete problem to P.

Exercise: Reduction of 3-SAT to CLIQUE.

- Input: a boolean formula $F(x)$ on n binary variables $x_1, ..., x_n$ in conjunctive normal form (conjunction $\wedge$ of $\vee$ clauses) with at most three literals by clause (x_i or $\bar{x}_i$)
- Output:
- `yes` if there is an $x \in \{0, 1\}^n$ satisfying F, and `no` otherwise.

2.2.4 Optimisation problems

A decision or optimisation problem is $\mathcal{NP}$-hard if any decision problem P' in $\mathcal{NP}$ reduces polynomially to P .

If we can solve the optimisation problem

$$\min_x c(x) \quad \text{s.t.} \quad x \in X$$

we can solve its decision version

$$\exists ?\ x \quad \text{s.t.} \quad x \in X \quad \text{and} \quad c(x) \leq c_0$$

To prove that an optimisation problem O is $\mathcal{NP}$-hard, it is enough to reduce a known $\mathcal{NP}$-complete decision problem to its decision version $\text{dec}(O)$.

Homework